Todays Content

1. Heaps Intro
2. Insert/delete()
3. Heapify()
4. Problems on Heap

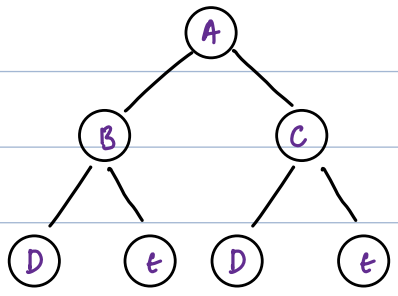
[Saturday :]

CBT Complete Binary Tree.

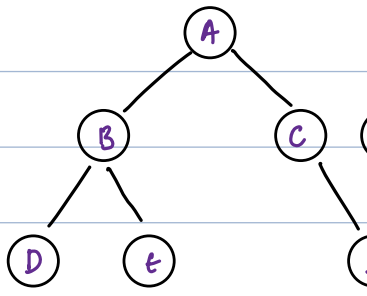
→ {Last level may or may not be filled}

All levels are filled [except possibly] the last level,
which is filled left → right.

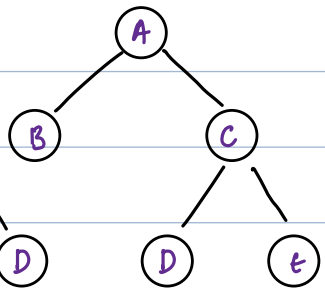
Ex1: CBT



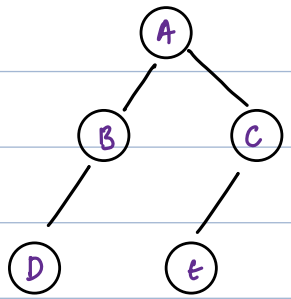
Ex2: Not CBT



Ex3: Not CBT



Ex4: Not CBT



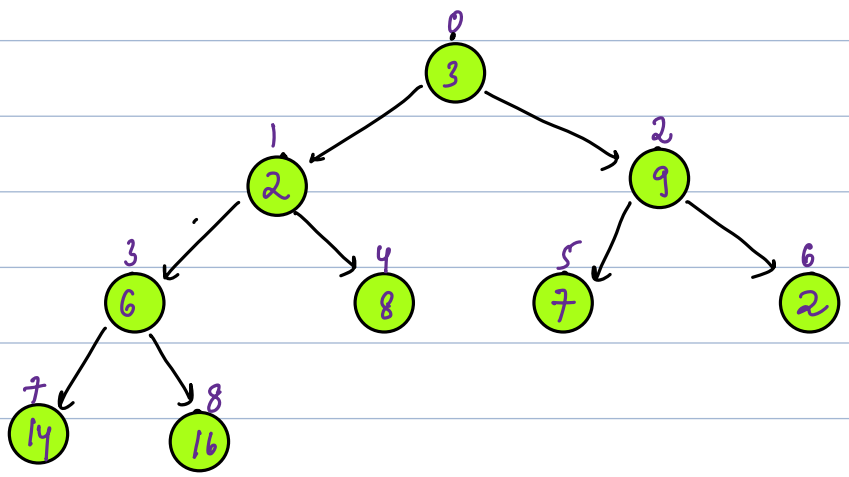
Height of CBT: $\log N$ N = No: of Nodes. → {Stay back}

Implementation of CBT using arrays / List

Insert: 3 2 9 6 8 7 2 14 16 → Implement a CBT:
↓ Insert data in a list.

	0	1	2	3	4	5	6	7	8
list:	3	2	9	6	8	7	2	14	16

Parent	left	Right
0	1	2
1	3	4
2	5	6
3	7	8
i	2i+1	2i+2



obs:

	left	Right
parent i :	2i+1	2i+2
child i :	(i-1)/2	

Advantage:

parent index → We can access child index
child index → We can access parent index
It allows both Top down & down Top.

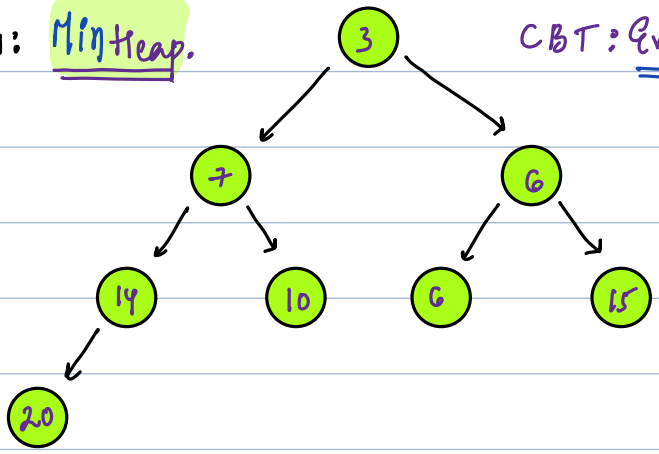
Heaps: a. MinHeap b. MaxHeap

1. BT should be a CBT first

2a. Every node <= child nodes → MinHeap

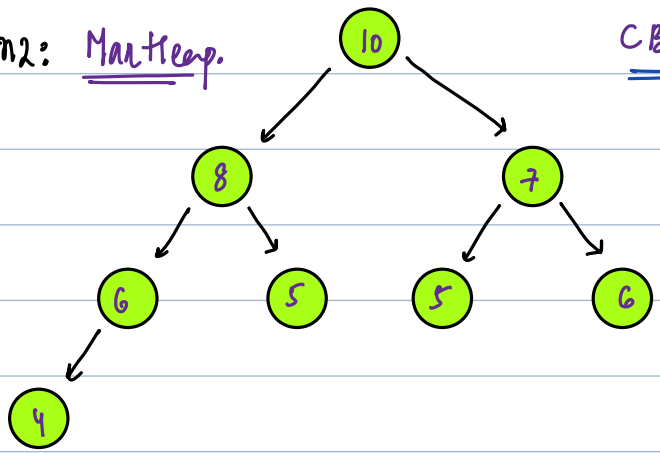
2b. Every node >= child nodes → MaxHeap

Eg1: MinHeap.



CBT: Every node <= child nodes

Eg2: MaxHeap.



CBT: Every node >= child nodes

Heap Operations.

MinHeap	MaxHeap	Time Complexity for Single
insert()	insert()	O(log N) // N: No of Nodes
getMin()	getMax()	O(1)
deleteMin()	deleteMax()	O(log N) // N: No of Nodes
size()	size()	O(1)
heapify()	heapify()	O(N) // N: no of Nodes

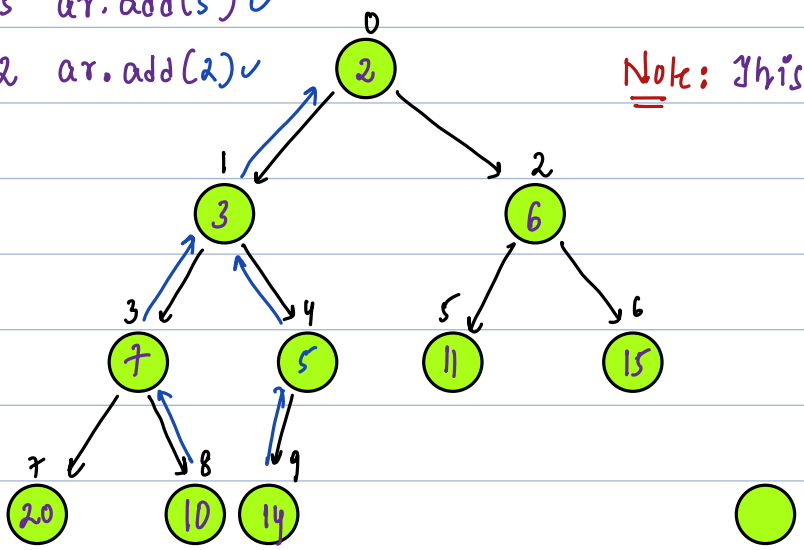
Heapify: Using given data create MinHeap/MaxHeap

Given min heap :

list : ar	0	1	2	3	4	5	6	7	8	9
	2	3	6	7	5	11	15	20	10	14

Insert : 5 ar.add(5) ✓

Insert : 2 ar.add(2) ✓



Note : This tree is our imagination

i-1/2

Insert(5) :

ind	par	Ideally : ar[par] <= ar[ind]
8	3	ar[3] <= ar[8] : swap ar[3] & ar[8]
3	1	ar[1] <= ar[3] : swap ar[1] & ar[3]
1	0	ar[0] <= ar[1] : break it.

i-1/2

Insert(2) :

ind	par	Ideally : ar[par] <= ar[ind]
9	4	ar[4] <= ar[9] : swap ar[4] & ar[9]
4	1	ar[1] <= ar[4] : swap ar[1] & ar[4]
1	0	ar[0] <= ar[1] : swap ar[0] & ar[1]

0 : at root, stop it.

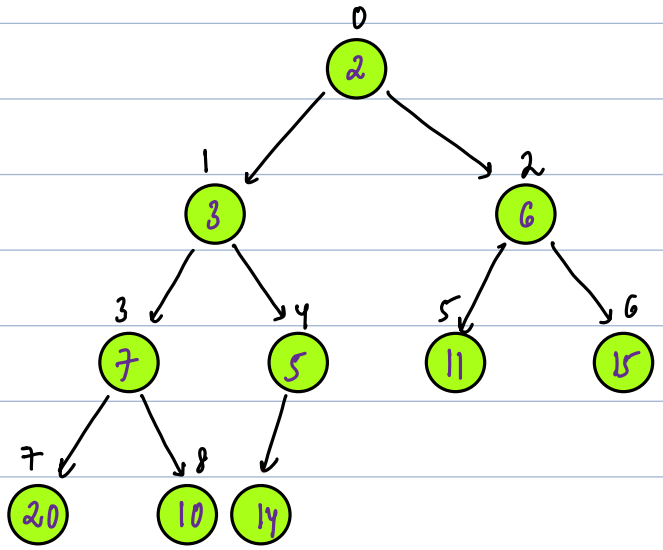
Note : For a single insert, we travel from leaf node to root node, dist = height

In CBT height = $\log_2^N$, No: of nodes.

Delete_Min():

deleted.

0	1	2	3	4	5	6	7	8	9
2	3	6	7	5	11	15	20	10	14



Step: Copy last element to start & delete last.

ind	left	right	min_ind
0	ar[]	ar[]	ar[], swap ar[] & ar[]

ind	left	right	min_ind
0	ar[]	ar[]	ar[], swap ar[] & ar[]

Heapify():

Given an arr[N] of elements, convert them into MinHeap.

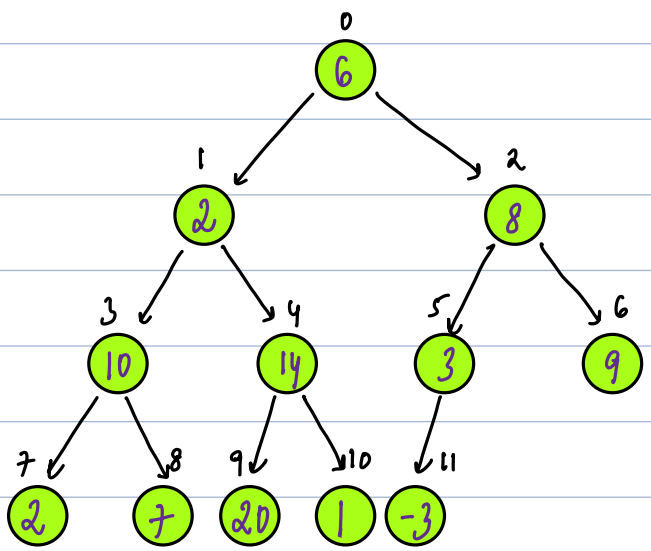
	0	1	2	3	4	5	6	7	8	9	10	11
Eg: arr[12] = {	6	2	8	10	14	3	9	2	7	20	1	-3 }

Idea:

Idea2: Add arr elements in arraylist as it is & apply

Heapify: Iterate in right to left, at every index, make that entire subtree a min_heap by comparing ele with it's children.

	0	1	2	3	4	5	6	7	8	9	10	11
list: {	6	2	8	10	14	3	9	2	7	20	1	-3 }



```
class Min_Heap {
    private ArrayList<Integer> ar;
    Min_Heap() { //constructr.
        ar = new ArrayList<Integer>();
    }

    void delete_Min() { TC: O(logN)

    }
    int getMin() { TC: O()

    }
```

void insert(int n){ TC: O( )

}

void heapify(int a[]){ TC: O( ) {ar[N] → Heap : TC: O( )}

}

}

PriorityQueue : Arranges data based on their priority values

Functions:

1. add( ) : add element in PriorityQueue : log(N) : N = no of elements
2. peek() : return peek element : O(1) : peek : highest priority
3. remove() : removes peek element : logN : N = no of elements
4. size() : returns size : O(1)

minPQ: Priority based on minvalues : {highest Priority}

```
PriorityQueue<Integer> pq = new PriorityQueue<>(); { MinHeap }
pq.add(19);
pq.add(10);
pq.add(15);
pq.add(18);
Print(pq.peek());:
pq.remove().
Print(pq.peek());:
Print(pq.peek());:
```

pq:

Note: If want own ordering, overide comperator

Added Code Link: Please refer it.

Problem : Connecting Ropes:

We are given an array that represents different size of ropes. In a single operation, you can connect two ropes.
Cost of connecting two ropes is sum of length of ropes you are connecting.
Find the minimum cost of connecting all the ropes into 1

Eg ar[ ] = { 2 5 3 2 6 } → Cost:

{ } → Total Cost:

Eg ar[ ] = { 2 5 3 2 6 } → Cost:

{ } → Total Cost:

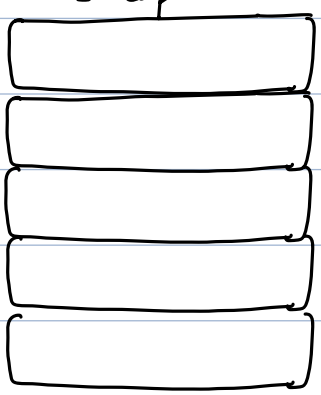
Idea:

Operations:

Data structures:

	0	1	2	3	4
ar[ ] = {	2	5	3	2	6 }

Min_Heap



1st getMin(): deleteMin()

2nd getMin(): deleteMin()

Insert

Total Cost =

```
int Min_Cost(int arr[]){

}
```